Contents

1 A Host-Driven Field Runtime for Portable Interface Behavior **1**
- 1.1 Abstract . . . 2
- 1.2 1. Introduction . . . 2
  - 1.2.1 1.1 The problem: behavior fused to the renderer . . . 2
  - 1.2.2 1.2 The host-driven answer . . . 2
  - 1.2.3 1.3 Contributions . . . 3
- 1.3 2. Background and related work . . . 3
- 1.4 3. The host boundary (the core/platform split) . . . 4
  - 1.4.1 3.1 Two packages, one direction . . . 4
  - 1.4.2 3.2 The `FieldHost` interface . . . 4
  - 1.4.3 3.3 `browserHost()`: one implementation among possible many . . . 5
  - 1.4.4 3.4 The proof: an empty-allowlist boundary test . . . 5
  - 1.4.5 3.5 Precision: what the boundary does and does not claim . . . 6
- 1.5 4. The frame scheduler and six registries (the DOM-participation contract) . . . 6
  - 1.5.1 4.1 Six ordered phases . . . 6
  - 1.5.2 4.2 Read/write discipline and the read-phase guard . . . 7
  - 1.5.3 4.3 Six single-concern registries . . . 7
  - 1.5.4 4.4 Correctness fix: relationship resolution is now real . . . 8
  - 1.5.5 4.5 `lintPlatform()`: the platform's self-audit . . . 8
- 1.6 5. The pattern runtime and conformance gates (portable, auditable behavior) . . . 8
  - 1.6.1 5.1 `applyRecipe`: a pattern compiled onto the platform . . . 9
  - 1.6.2 5.2 `validateRecipe`: a program cannot reference vocabulary the engine lacks . . . 9
  - 1.6.3 5.3 Passports cross-checked against ground truth, proven headless . . . 10
  - 1.6.4 5.4 Supplied-vs-derived discipline: confidence is never fabricated . . . 10
- 1.7 6. Portability evaluation (an engineering argument) . . . 11
  - 1.7.1 6.1 What the architecture buys . . . 11
  - 1.7.2 6.2 The evidence . . . 11
  - 1.7.3 6.3 Honest limits . . . 11
- 1.8 7. Limitations . . . 12
- 1.9 8. Discussion . . . 12
- 1.10 9. Conclusion . . . 13
- 1.11 Appendix A. Reproducibility . . . 13
- 1.12 Appendix B. Conversion notes (markdown → preprint) . . . 14
- 1.13 Citation coverage . . . 14

1 A Host-Driven Field Runtime for Portable Interface Behavior

Status: research draft (preprint, work in progress). Paper 5 of the Fundamental family — the systems-architecture paper. Claims verified against the codebase and canonical docs as of 2026-06-26. See the series index and *the caveat canon* therein. This is a preprint draft, not canonical product documentation.

Post-verification note (0.8.1). Three portability additions landed after the original draft and are reflected above where load-bearing: `registerOverlay()` (a public hook for registering custom overlays), the wire-format contract (`PARTICLE_STRIDE` / `PARTICLE_WIRE_VERSION` plus `readParticleChannels`) for transporting particle state across a worker/process boundary, and `headlessHost()` — a DOM-free reference host exported from `@fundamental-engine/core` (#600), which turns the "alternative host" of §3.3 and §6 from hypothetical into shipped.

Author: Zach Shallbetter

Series: Fundamental Research Papers, Paper 5 of 8 (the runtime-architecture paper) **Companion paper (flagship):** Fundamental: A Field Translation Runtime for Relational DOM Interfaces — the paradigm, the reciprocal model, and the architecture in overview. This paper goes *deep* where the flagship was an overview; it does not restate the model.

1.1 Abstract

UI-behavior systems usually fuse computation with the render target: the logic that decides *what an interface does* reaches directly for `document`, `window`, `requestAnimationFrame`, and a 2D canvas, so the behavior cannot run headless, cannot be retargeted to a different renderer, and cannot be tested without a browser. A relational *field* model — particles, forces, and a continuously sampled density field — is especially exposed to this failure, because the obvious implementation is canvas-bound by construction. This paper presents the runtime architecture by which **Fundamental** avoids it: a renderer-agnostic *core* that computes field behavior against plain data, a browser *platform* that owns DOM participation, and a single injected **host boundary** (`FieldHost`) that is the only seam between them. The core's freedom from the render target is not asserted but *enforced*: a boundary test scans every core source file for DOM globals and runs with an **empty allowlist**, so the core imports no DOM globals and computes with no document present. On top of that boundary sit two contracts that make field participation portable and auditable: a six-phase frame scheduler (`discover → read → compute → state → write → render`) with six single-concern registries that eliminate read-after-write layout thrashing, and a pattern runtime whose programs are validated against the engine's real, passported vocabulary and proven by a headless conformance harness that runs *the same physics* the live product does. We argue the engineering case for portability — the same core already drives a canvas surface, SVG overlays, CSS-variable feedback, and a headless test harness, and could target WebGL, native, or a different document through a custom host — and we are candid about the one place the boundary is not yet clean: a legacy element write-back path in `engine/field.ts` is still being migrated behind the platform registries (Phase 5; tracking issue #228). Portability here is demonstrated by a *proven boundary plus host injection*, not by a second shipping production renderer.

1.2 1. Introduction

1.2.1 1.1 The problem: behavior fused to the renderer

Most interface-behavior code is *written against the browser*. A component's interaction logic calls `element.getBoundingClientRect()` inline, reads `window.innerWidth`, schedules work with `requestAnimationFrame`, instantiates a `ResizeObserver`, and — in any system that draws — talks to a 2D canvas context. This fusion is convenient and, for a single target, harmless. It becomes a problem the moment one asks a different question: *can this behavior run without a browser?* The usual answer is no. The logic cannot be unit-tested without a DOM, cannot be moved to a different render surface without rewriting it, and cannot be audited in isolation because it is interleaved with the very globals it manipulates.

A *field* model raises the stakes. Fundamental treats the interface as a shared relational field — elements exert force, particles move, and a density signal is written back to the DOM as CSS custom properties and thresholded events (the reciprocal loop developed in the flagship, §3). The naive implementation of such a model is a particle canvas: a `<canvas>` element, a 2D context, a render loop, and force math entangled with all three. That implementation would be *canvas-bound* — the field would be inseparable from one specific way of drawing it, and the claim that Fundamental is "a relational field runtime, of which the canvas is one render surface" (flagship §1.2) would be rhetorical rather than structural.

1.2.2 1.2 The host-driven answer

Fundamental's answer is to separate *what the field does* from *how the browser participates in it*, and to let the two communicate through exactly one injected interface:

- A renderer-agnostic **core** (`@fundamental-engine/core`) computes field, force, particle, metric, diagnostic, and conformance logic against plain data. It imports no DOM globals.
- A browser **platform** (`@fundamental-engine/dom`) owns DOM participation: measurement, state, feedback, relationships, visual bindings, overlays, scheduling, and linting.

- A single **host boundary** — the `FieldHost` interface — is the only path by which the core reaches the environment (viewport, scroll, rAF, reduced-motion, visibility, scan root, events, a canvas factory). The platform supplies one implementation, `browserHost()`; any other implementation drives the *same* engine from a different environment.

Because the seam is a single injected interface rather than a scatter of inline globals, "renderer- agnostic" stops being a design aspiration and becomes a property that can be *checked* — and, in this codebase, is checked on every test run.

1.2.3 1.3 Contributions

This paper contributes the *runtime architecture and its conformance methodology*:

1. **The layered host-driven architecture and its proven boundary** (§3): the core/platform split, the `FieldHost` interface, the strict one-way dependency direction, and the empty-allowlist boundary test that converts portability into a continuously checked invariant — stated precisely, including the one legacy path still being migrated.
2. **The six-phase scheduler and six-registry design as a DOM-participation contract** (§4): how ordered phases and single-concern registries make DOM participation a disciplined, lintable protocol rather than ad hoc glue, and why that discipline is what lets a field ride the browser's layout instead of fighting it.
3. **The pattern runtime and conformance gates as portable, auditable behavior** (§5): how a pattern compiles onto the platform, how validation rejects programs that reference vocabulary the engine does not have, and how the passport live-registry conformance-catalog cross-check and the headless physics harness make a behavior's claims checkable across host targets.

We then make the engineering case for portability (§6), state the limitations honestly (§7), discuss why host-driven separation matters for UI-behavior systems generally (§8), and conclude (§9). This paper is a **systems paper**: it presents no user study. Its "evaluation" is the portability-plus-conformance argument of §6. Everything else — the paradigm (Paper 1), reading (Paper 2), evidence and trust (Paper 3), accessibility (Paper 4), recipe-authoring detail (Paper 6), data binding (Paper 7), and diagnostics (Paper 8) — is deferred to its own paper.

1.3 2. Background and related work

Fundamental's runtime architecture draws on several established lines of systems and UI work; we position it against each and cite external references through verified keys in the family bibliography.

Retained-mode vs immediate-mode UI. The retained/immediate distinction frames how a system relates computation to drawing: a retained-mode toolkit keeps a persistent object model and redraws from it; an immediate-mode UI re-issues draw calls from application state each frame. Fundamental is neither, exactly — it keeps a persistent *field* (bodies, agents, relationships, a scalar grid) computed in the core, and treats every visible surface (canvas, SVG overlays, CSS feedback) as an *immediate* read of that persistent state. The separation of a persistent model from its rendering is the lineage; the novelty is that the model is a relational field and the rendering targets are plural. [retained-immediate-ui]

Renderer-agnostic and headless UI. A renderer-agnostic engine computes against an abstract scene and binds to a backend (DOM, native, a test renderer) late; headless UI libraries provide behavior and state without prescribing markup or pixels. Fundamental's core is renderer-agnostic in the strong sense — it is *mechanically prevented* from importing a render target — and its conformance harness is a headless consumer of the same engine. [react-test-renderer; headless-ui]

The host/adapter (ports-and-adapters) pattern. Hexagonal architecture isolates a domain core behind *ports* (interfaces the core defines) implemented by *adapters* (environment-specific code). `FieldHost` is precisely a port: the core declares the interface it needs from any environment, and `browserHost()` is the

browser adapter. The contribution here is not the pattern but its *enforcement* in a UI-behavior runtime, where the temptation to reach for a global is constant. [cockburn2005]

Reactive runtimes and the scheduling of reads and writes. Modern UI runtimes schedule work to avoid layout thrash — batching DOM reads and writes so that a measurement never forces a reflow against a pending mutation. Fundamental makes this scheduling discipline a first-class, *named* phase contract (§4) and a lint rule, rather than an internal optimization. [fastdom; webdev-layout-thrashing]

Testability of UI logic. A recurring argument in UI engineering is that behavior is testable in proportion to how cleanly it is separated from the renderer. Fundamental takes the strong form of that position: the boundary that makes the core portable is the same boundary that makes it testable, and the conformance harness exercises the production physics headlessly (§5.3). [fowler2004presentation; meszaros2007]

The distinguishing stance, across all of these, is that Fundamental treats the renderer-agnostic boundary and the participation contract as *auditable artifacts* — a test with an empty allowlist, a phase-discipline lint, a pattern validator, and a passport cross-check — not as conventions the authors promise to honor.

1.4 3. The host boundary (the core/platform split)

1.4.1 3.1 Two packages, one direction

Fundamental's runtime is split across packages whose dependency direction is strict and one-way (the package hierarchy is given in `docs/canonical/platform-architecture.md`):

```
@fundamental-engine/core  renderer-agnostic field / force / particle / metric / diagnostic / conformanc
                    Computes field behavior against plain data. Imports no DOM globals.
@fundamental-engine/dom  DOM participation: measurement, state, feedback, relationships, visual binding
                    overlays, scheduling, linting - plus the browser host adapter.
@fundamental-engine/elements  <field-root> / <field-cell> custom elements + the [data-body] authoring c
@fundamental-engine/react     <FieldField> component + useFieldField hook.
@fundamental-engine/vanilla   the framework-free FieldField class.

elements → platform → core      react → platform → core      vanilla → platform → core
```

The direction encodes the central claim: *field behavior is independent of its presentation substrate.* The core never imports the platform; the platform depends on the core only for contracts and types. Every authoring surface (`elements`, `react`, `vanilla`) is an adapter over the platform, never a peer of the core. This is the flagship's §4.1 layering; here we are concerned with the seam it turns on.

1.4.2 3.2 The `FieldHost` interface

The core reaches the environment through exactly one injected interface, `FieldHost` (`packages/core/src/engine/host.ts`). It is *pure types — no globals* — so the file that imports it imports no DOM:

```ts
export interface FieldHost {
  root: ParentNode;                          // the subtree scanned for [data-body] / [data-on] / …
  viewport(): HostViewport;                  // { width, height, dpr }, read each resize
  scrollY(): number;
  scrollHeight(): number;                    // reading it forces reflow - so the caller caches it
  reducedMotion(): boolean;                  // freezes the sim
  hidden(): boolean;                         // backgrounded tab → pause the loop
  raf(cb: (t: number) => void): number;      // schedule the next frame
  cancelRaf(id: number): void;
  createCanvas(): HTMLCanvasElement;         // an offscreen canvas for the heatmap buffer
  onResize(cb: () => void): () => void;      // subscribe; returns an unsubscribe
  onScroll(cb: () => void): () => void;
```

```
  onVisibility(cb: () => void): () => void;
  onInput(cb: () => void): () => void;      // pointer / wheel / key / touch activity
  onBodyEvent(type: string, cb: (e: Event) => void): () => void;
}
```

Every environment touchpoint the engine could possibly want — viewport size and device-pixel ratio, scroll position and page height, reduced-motion and visibility preferences, frame scheduling, a canvas factory for the heatmap buffer, and the event subscriptions for resize, scroll, visibility, input, and composed shadow-DOM body events — is a *method on the host*, not a global the engine calls. The engine's entry point, `createField(canvas, opts)`, makes the dependency mandatory: it throws if `opts.host` is absent (`packages/core/src/engine/field.ts`), so there is no silent fallback to `window`.

1.4.3 3.3 `browserHost()`: one implementation among possible many

The platform supplies the browser implementation, `browserHost()` (`packages/dom/src/browser-host.ts`). It is the single place `window`, `document`, `requestAnimationFrame`, and `matchMedia` are wired to the engine's needs:

```
export function browserHost(): FieldHost {
  return {
    root: document,
    viewport: () => ({ width: window.innerWidth, height: window.innerHeight, dpr: window.devicePixelRati
    scrollY: () => window.scrollY || 0,
    scrollHeight: () => document.documentElement.scrollHeight,
    reducedMotion: () => matchMedia('(prefers-reduced-motion: reduce)').matches,
    hidden: () => document.hidden,
    raf: (cb) => requestAnimationFrame(cb),
    // … onResize / onScroll / onVisibility / onInput / onBodyEvent → addEventListener …
  };
}
```

Because every DOM global lives behind this one function, an *alternative* host — a headless harness, a different document, a native shell, a WebGL or WebGPU surface — drives the same engine by supplying a different object with the same shape. `createBrowserField()` is the convenience that pairs `createField` with `browserHost()`; both live in the platform, never the core.

1.4.4 3.4 The proof: an empty-allowlist boundary test

The renderer-agnostic claim is not asserted; it is *enforced* by `packages/core/src/engine/dom-boundary.test.ts`, the architectural keystone. The test walks every non-test source file in `Fundamental` and fails if any of them contains a DOM-global *call-site*, matched as access/construction patterns so that ordinary prose ("scan the document", "debounce window") does not trip it:

```
const ALLOW = new Set<string>();   // empty: no core file may touch a DOM global

const FORBIDDEN: Array<[string, RegExp]> = [
  ['document.<member>', /\bdocument\.(querySelector|createElement|documentElement|body|head|addEventLis
  ['window.<member>',   /\bwindow\.(innerWidth|innerHeight|addEventListener|scrollY|matchMedia|getCompu
  ['rAF/timers/media',  /\b(requestAnimationFrame|cancelAnimationFrame|getComputedStyle|matchMedia)\s*\
  ['new <Observer>',    /\bnew\s+(ResizeObserver|IntersectionObserver|MutationObserver)\b/],
];
```

The decisive line is `const ALLOW = new Set<string>()` — an **empty allowlist**. There is no file in the core that is permitted to touch a DOM global; a single new call-site, anywhere in the core, fails the test. A second test guards any future allowlisted module; with the allowlist currently empty its loop is a no-op, so today's anti-rot protection comes entirely from the first test — any new call-site fails it. The practical

consequence: *the core imports no DOM globals and computes field behavior with no document present*, which is exactly what makes it portable to any renderer through a custom host.

1.4.5 3.5 Precision: what the boundary does and does not claim

The caveat canon obliges precision here, because an over-claim would be easy and wrong.

What is true. The core imports *no DOM globals*. Every environment touchpoint — viewport, scroll, rAF, reduced-motion, visibility, the scan root, events, the canvas factory — goes through the injected `FieldHost`. The boundary test runs green with an empty allowlist, and the core's logic runs in the headless conformance harness with no browser present (§5.3).

What is *not* claimed. The core still *operates on DOM nodes that are injected into it*: the body scanner walks the host-supplied `root`, and a body's geometry comes from a `getBoundingClientRect()` on an element the host handed in. Operating on an injected node is not the same as reaching for a global, and the boundary test (which forbids `document.*` / `window.*` / `requestAnimationFrame` / `new ResizeObserver`, not "all element access") correctly distinguishes the two.

The one legacy path. A *legacy element write-back path in `engine/field.ts` is still pending migration.* The engine's canvas simulate-and-render loop writes feedback directly onto bodies it was handed — CSS custom properties (`el.style.setProperty('--field-density', …)`), a `transform` on moving layouts, and `data-*` flags — rather than routing every such write through the `FeedbackRegistry`. Phase D added a `feedbackSink` seam so those writes *can* be routed to the platform, and the platform runtime is the default for `<field-root>`; but the legacy path remains as the canvas surface's own write-back and is tracked for migration behind the platform registries (Phase 5; tracking issue #228). This is element write-back on injected nodes, **not** a DOM global — so it does not break the empty-allowlist boundary — but it does mean the platform does not yet own *all* DOM writes. We therefore do **not** claim "the platform owns all DOM writes." The canonical contracts state the same thing plainly:

> `Fundamental` is renderer-agnostic and imports no DOM globals (a legacy element write-back path still lives in `engine/field.ts`, pending migration). — `system-contracts.md` §24

Stating this boundary precisely is part of the contribution: the value of an *enforced* boundary is that it lets the paper say exactly how clean the separation is, and exactly where it is not yet clean, without hand-waving.

1.5 4. The frame scheduler and six registries (the DOM-participation contract)

The host boundary makes the core portable. The platform then needs its own discipline, because DOM participation done carelessly thrashes layout. Fundamental makes participation a *contract*: an ordered phase pipeline and six single-concern registries, each declaring the one phase it runs in. The flagship gives the overview (§5); here we go through the mechanism and the guarantees it buys.

1.5.1 4.1 Six ordered phases

The platform runs one shared loop whose phases are fixed and ordered (`packages/dom/src/schedule.ts`):

```
discover → read → compute → state → write → render
```

1. **discover** — register/unregister bodies, relationships, and visual bindings (structure changes).
2. **read** — measure the DOM once: snapshot every registered element's geometry and visibility. *This is the only place layout is read.*
3. **compute** — run field/force/agent logic against the immutable snapshot. No DOM access.
4. **state** — fold results into the state registry and apply thresholds. Internal truth, no DOM writes.
5. **write** — flush state to CSS custom properties, data attributes, `ElementInternals` state, and thresholded events.
6. **render** — draw overlays, field lines, and heatmaps from the registries (read-only).

The `FrameScheduler` is a small, DOM-free state machine: `runFrame(now, viewport)` walks the six phases in order, calls each phase's handlers, and returns a `FrameReport` listing which phases ran and any violations. The scheduler "holds no DOM and is fully testable" — frames are deterministic because `now` and `viewport` are injected, so an entire frame can be driven in a unit test with no browser.

1.5.2 4.2 Read/write discipline and the read-phase guard

The point of the ordering is a single invariant: *no DOM writes during read or compute; no geometry reads during write.* The scheduler enforces it. `READ_PHASES` is `['discover', 'read']`, and `assertPhase(allowed, op)` records (or, under `strict`, throws) a `PhaseViolation` whenever an operation runs in the wrong phase. The scheduler exposes `readGuard()`, a function the platform hands to the `MeasurementRegistry` via `setPhaseGuard`. The registry consults the guard before reading layout:

```
measure(now = 0, viewport?: Viewport): readonly FieldMeasurement[] {
  this.guard?.('measure');  // read-phase discipline: reading layout off-phase thrashes
  // … read every registered element's box once, return an immutable snapshot …
}
```

Crucially, the registry does *not* import the scheduler — the coupling is a single injected function, mirroring the host pattern one layer down. Outside a managed frame (`phase === null`, e.g. a direct unit-test call) the guard is permissive; inside a frame an off-phase read is caught. The payoff is the elimination of read-after-write tearing and forced reflow: the loop reads *all* geometry once into one immutable snapshot, computes against it, and flushes *all* writes together. This is what makes the synchronization invariant of the flagship (§3.2) cheap to maintain even as the DOM animates — the field rides the browser's layout instead of fighting it.

1.5.3 4.3 Six single-concern registries

Each registry owns exactly one kind of DOM participation and one phase (`packages/dom/src/{measurement,state,feedbac`

Registry	Concern	Phase	Key discipli
`MeasurementRegistry`	frame-stable geometry/visibility for bodies	read	one snapsho
`StateRegistry`	typed, observable element state (number/bool/string/vector2)	compute / state	*internal trut*
`FeedbackRegistry`	CSS-variable + thresholded-event write-back	write	the *single w*
`RelationshipRegistry`	typed graph of connections between bodies	compute	normalizes r
`VisualBindingRegistry`	binds expressive visual layers to semantic sources	write	a decorative
`OverlayRegistry`	relationship/field-line/diagnostic overlays	render	render-only;

Three properties are worth drawing out, because they are what make participation *safe*.

First, the `StateRegistry` holds internal truth that is explicitly *not* accessibility state, and the `FeedbackRegistry` is the only module permitted to turn that state into DOM-visible output. A single writer for CSS variables avoids the races and stale values that plague systems where any handler can poke at `element.style`. The feedback flush writes a bound state value to its CSS variable (the legacy `--forces-*` CSS variables have been removed); threshold events fire only on enter/exit edges via a `Thresholder` with hysteresis and debounce, so there are no noisy per-frame DOM events, and they still emit `forces:*` event aliases for compatibility.

Second, the `RelationshipRegistry` encodes the observation that *the DOM is a tree, but interfaces are graphs.* Rather than invent a parallel graph, it normalizes the relationships the platform *already expresses* — `a[href#id]`, `label[for]`, `aria-controls` / `-describedby` / `-labelledby` / `-flowto`, and `data-field-relation` / `-target` — into one typed relationship graph mapped onto core `RelationshipAgents`. Native semantics are respected first; Fundamental does not duplicate links the HTML already declares.

Third, the `OverlayRegistry` and `VisualBindingRegistry` enforce that *overlays reveal; they do not define.* An overlay is a render record that resolves to geometry from the measurement snapshot; it owns no relationship and mutates no physics. A visual binding declares that an expressive layer *represents* a semantic source without becoming the sole carrier of its meaning — the accessibility seam developed in Paper 4.

1.5.4 4.4 Correctness fix: relationship resolution is now real

A relational graph is only honest if it distinguishes a link that *resolves* from one that points at nothing. The `RelationshipRegistry` now tracks both. `scanRelationships(el, resolve)` partitions an element's declared edges into *resolved* (both endpoints known) and *unresolved* (an id-ref that resolves to no element), and the registry keeps the unresolved declarations rather than silently dropping them:

```
const edge = (idref, type, source, strength) => {
  const t = resolve(idref.replace(/^#/, ''));
  if (t) resolved.push(rel(el, t, type, source, strength));
  else  unresolved.push({ from: el, type, target: idref, source });  // counted, not dropped
};
```

This matters downstream: a pattern's metric computation can now report a *real* resolution ratio — `resolved / (resolved + unresolved)` — so a citation that points at nothing lowers coherence and raises entropy instead of being invisible (the fix merged in #222; see §5.4). A subtle bug the fix also closed: id-less elements were collapsed onto one fallback key (`tagName`), so edges keyed by endpoint id collided and overwrote each other; a `WeakMap` now hands each id-less element a stable, unique `el-<seq>` id on first sight. Resolution is now a property the graph can be *audited* on, not a number hardcoded to look perfect.

1.5.5 4.5 `lintPlatform()`: the platform's self-audit

A field layer fails *quietly* — a relation points at a missing id, a decorative visual duplicates text a screen reader already reads, an element is styled from state it was never registered for. None of these are type errors; all are data pathologies. `lintPlatform(platform)` (`packages/dom/src/lint.ts`) aggregates pure, read-only guardrail rules that surface them:

```
relation-target-missing      a [data-field-relation] points at a missing/unresolvable body
state-unregistered           an element holds field state but was never registered for measurement
overlay-without-links        a relationship overlay has no relationships behind it
feedback-non-css-var         a binding writes ARIA/attributes instead of a --field-* variable (error)
measurement-off-phase        layout was read outside the read phase
visual-orphan                a representation visual binds no semantic source (error)
visual-not-hidden            a decorative visual is not aria-hidden
```

Each rule is a pure function over the registries (or the DOM, given a resolver), so it is testable without a live page, and `lintPlatform` runs them over a constructed platform, reporting violations by rule name. The `feedback-non-css-var` and `visual-orphan` rules are *errors*, not warnings: writing state into ARIA, or shipping a representation with no semantic source, are correctness/accessibility hazards, not style nits. The non-negotiable property: *lint reads; it never mutates state, physics, or the DOM.* It is the platform's self-audit, run in the same spirit as the conformance suite for forces (§5) — a static check that the participation contract is actually being honored.

1.6 5. The pattern runtime and conformance gates (portable, auditable behavior)

The host boundary makes the engine portable; the scheduler makes participation safe. The third leg of the architecture makes *behavior itself* portable and auditable: a pattern is a serializable field program, and the runtime that applies it — together with the validation, passport, and conformance machinery — guarantees that a program only ever references vocabulary the engine actually has, and that the behavior behind that

vocabulary is proven by tests that run the production physics. Paper 6 covers pattern *authoring*; this section covers the *runtime and conformance* that make patterns a portable, auditable behavior format.

1.6.1 5.1 `applyRecipe`: a pattern compiled onto the platform

`applyRecipe(root, pattern, options)` (`packages/dom/src/apply-recipe.ts`) is the DOM counterpart to the core's `compileRecipe`. It turns a `FieldPattern` record into a running field program on a scoped `createFieldPlatform(root)` — the registry/feedback layer, *not* a particle canvas — so it runs on ordinary content the way the Reading Field studies do. The sequence is:

1. **Validate, then compile.** `validateRecipe(pattern)` runs first; an invalid pattern throws before anything touches the DOM. The valid pattern is compiled to bodies, a feedback variable map, metric lanes, and a reduced-motion description.
2. **Register bodies (token lane only).** Each body element is registered for measurement and annotated with the pattern's `data-body` tokens. Only the *token lane* is executed — concept words are never run as forces.
3. **Bind the metric lane to feedback variables.** Each metric maps to a `--field-*` variable through the `FeedbackRegistry`, so the write phase is the single writer.
4. **Discover relationships once**, on the `discover` phase.
5. **Compute metrics each frame** (`compute` → `state`), then let the `write` phase flush state to the bound CSS variables.
6. **Install a real reduced-motion output** — an actual static `<aside>` describing meaning without motion, not just prose — when motion is reduced.

The returned handle is *inspectable and destroyable*: `inspect()` returns live measurement counts, the resolved/unresolved relationship split and resolution ratio, the per-element metric values, and the current lint count; `destroy()` clears every CSS variable the pattern wrote, removes created elements, restores overwritten attributes, and tears down the loop. Because the whole thing runs on a scheduler that can be driven by hand (`drive: false` lets a caller call `tick()` directly), a pattern's behavior can be exercised frame-by-frame in a test with no rAF and no browser.

1.6.2 5.2 `validateRecipe`: a program cannot reference vocabulary the engine lacks

The decisive property of the pattern format is its *conformance gate*. `validateRecipe` (`packages/core/src/recipes/schema.` rejects any pattern whose references are not real:

- **Every force token is a passported force.** Each token in every body is checked against `passportFor(token)`; an unknown token is a problem.
- **Declared primitives must equal the body tokens.** The pattern's `primitives` array must list *exactly* the distinct tokens its bodies use — no drift between the human-facing declaration and the executable bodies.
- **Cross-lane guard.** A primitive must be a real runtime token, *never* a diagnostic, metric, condition, or concept. An `OTHER_LANE` map turns the generic "unknown token" error into a lane-aware one: `"'potential' is a diagnostic, not a runtime token"`, `"'mass' is a metric"`, `"'spring' is a concept"`. The map's keys are guaranteed *not* to be real tokens (`pressure`, `memory`, `gate` are real tokens and are deliberately absent), so the guard cannot misclassify a genuine force.
- **Every render layer and diagnostic is a known mode**, and the natural field, tier, and status are each one of their enumerated values.
- **An accessibility fallback is required** — `reducedMotion` and `meaningWithoutMotion` must both be present, so *no pattern is motion-only* (the accessibility invariant of Paper 4, enforced at the authoring boundary).

The effect is that a pattern is *auditable against the engine's actual capabilities.* Pattern prose may be expressive ("completion releases pressure and decays into memory") while the runtime fields stay strict (`primitives: ['morph', 'memory', 'gravity']`), and a concept word can never silently invent a force. The shipped catalog (the records in `packages/core/src/recipes/catalog.ts`, re-exported by `gallery.ts`) is itself a conformance fixture — a test runs every pattern through `validateRecipe` (`recipes/schema.ts`):

every one of the 64 portable patterns passes `validateRecipe`, enforced by a test, so the catalog cannot drift from the engine's vocabulary.

1.6.3 5.3 Passports cross-checked against ground truth, proven headless

Behind every token a pattern may reference is a *passport* (`packages/core/src/contracts/passport.ts`): a machine-readable declaration of what a force is and does — its family and class, its truth mode, whether it owns a `field()`, whether it reads `env.fieldAt()`, whether it moves particles, does work, conserves speed, requires charge or velocity, or affects neutral matter. A passport is only trustworthy if it cannot drift from the force it describes, so `validatePassports(registry, catalog)` cross-checks the authored passports against *two* sources of ground truth:

- **The live registry.** `ownsField` must match whether `force.field` is actually a function; `canVisualizeFieldLines` must equal `ownsField`; `isSource` must equal `klass === 'S'`; `isModifier` must equal `klass === 'modifier'`.
- **The conformance catalog.** A force's declared `family` and `klass` must match what its conformance experiment says.

This cross-check runs as a test (`packages/core/src/contracts/passport.test.ts`: `validatePassports(allForces(), EXPERIMENTS)`), so a passport that claimed a force owns a field it does not, or sits in a class the conformance catalog contradicts, fails CI. The conformance experiments themselves are run by a *headless harness* (`packages/core/src/conformance/run.ts`) that simulates each scenario with the **real engine** — the same `FieldStore`, the same `step()` integrator, real neighbor queries, a real scalar grid advanced each frame, RNG made deterministic by a seeded PRNG. This is the architectural punchline of the host boundary: because the core imports no DOM globals, the *same physics the live product runs* can be run with no browser at all, and its claims checked exactly. The conformance suite asserts that every registered force has an experiment and that each experiment's expectations hold — so a behavior is proven once and the proof is portable to every host target, because the thing being proven is the renderer-agnostic core.

1.6.4 5.4 Supplied-vs-derived discipline: confidence is never fabricated

A portable, auditable behavior format is worthless if the metrics it computes invent evidence. The platform metric library (`packages/dom/src/metrics.ts`) draws a hard line between metrics it may *compute* and metrics it may only *receive*:

- **Computed** generically from observation: `attention`, `memory`, `recency` (proximity + engagement + decay) and `coherence`, `entropy`, `pressure`, `priority` (from relationship resolution and age).
- **Supplied-only:** `confidence` and `risk`. The engine has no evidence for a claim's truth — or for how dangerous it is — so each is present *only* when the host supplies it (via `data-field-confidence`, pattern options, or a domain trust/risk model). Neither is **ever** inferred from relationship presence, and `risk` is **never** defaulted to `0` — *a citation is not certainty, a source is not proof,* and "no risk" is a claim, not a safe blank.

This is a recent, deliberate correctness fix. The previous code computed `confidence: relTotal > 0 ? resolvedRatio : 0`, which meant "any citation implies full confidence" — exactly the wrong default for an evidence/trust surface (Paper 3). The fix (merged in #220) removes confidence from the computed set entirely; the `ComputedMetrics` type encodes the asymmetry by making `confidence` optional while every other lane is a required number. Relationship *resolution* is a separate, legitimately computed signal (§4.4) — and it, too, was made real in #222 (the metric computation now counts declared-but-unresolved edges toward the total, so resolution is honest). Together these fixes mean that what a pattern writes back to the DOM is either *observed*, *declared by the host*, or *honestly absent* — never fabricated. That property is what makes the behavior auditable across host targets: the same pattern, on the same core, behind any host, computes the same honest metrics.

1.7 6. Portability evaluation (an engineering argument)

This is a systems paper, so the "evaluation" is not a user study but an *engineering argument*: given the architecture above, what portability does it actually buy, and what evidence supports the claim?

1.7.1 6.1 What the architecture buys

One core, several render surfaces, today. The same renderer-agnostic core already drives multiple surfaces in the shipping system: the particle **canvas** (the legacy simulate-and-render loop), **SVG overlays** (relationship lines and field lines resolved by the `OverlayRegistry` from the measurement snapshot), and **CSS-variable feedback** (the `FeedbackRegistry`'s `--field-*` write-back, which drives weight, size, color, and glow in pure CSS). These are not three engines; they are three readings of one field. A pattern applied with `applyRecipe` runs entirely on the CSS-feedback surface — no canvas at all — which is the existence proof that the field is separable from any one way of drawing it.

A headless target, today. The conformance harness (§5.3) is a *fourth* consumer of the same core, and the most demanding one: it runs the production physics with no browser, no canvas, and no DOM, deterministically. A system whose physics can be executed headlessly is, by construction, a system whose physics is not fused to the renderer. As of 0.8.1 this is no longer only a test harness: a DOM-free reference host, `headlessHost()`, ships as a public export from `@fundamental-engine/core` (#600), so an alternative host is no longer hypothetical — the core ships with one.

A path to new targets. Because the only environment seam is `FieldHost`, a new render target — WebGL, WebGPU, a native shell, or a different document — is reachable by writing a new host, not by rewriting the engine. `createField` already requires the host and refuses to run without it, so the extension point is load-bearing rather than vestigial.

1.7.2 6.2 The evidence

The argument rests on three checkable artifacts:

1. **The empty-allowlist boundary test** (`dom-boundary.test.ts`, §3.4). The strongest single piece of evidence: the core is *mechanically prevented* from importing a DOM global, so portability is a continuously checked invariant rather than a claim that decays as the code grows.
2. **The deterministic test suite.** The system ships on the order of 560+ deterministic tests across the packages — the boundary guard, the scheduler and registry suites, the lint suite, the passport cross-check, and the headless conformance physics — all runnable with no browser. (Exact counts drift with the code; this is an order of magnitude as of the verification date, consistent with the flagship's reporting.)
3. **The multi-surface render modes.** The canvas, SVG-overlay, and CSS-feedback surfaces in the shipping product, plus the headless harness, demonstrate the same core under four different readings.

1.7.3 6.3 Honest limits

The portability claim is bounded, and we state the bounds plainly.

- **The legacy write-back is the one place the boundary is not fully clean.** As detailed in §3.5, the canvas surface's element write-back in `engine/field.ts` still writes CSS variables and transforms onto injected nodes directly rather than through the `FeedbackRegistry`. This is element write-back, not a DOM global, so it does not break the empty-allowlist boundary — but it is the reason we do not claim the platform owns *all* DOM writes. It is tracked for migration behind the registries (Phase 5; #228).
- **There is no shipping production renderer (WebGL, native) yet.** Portability here is demonstrated by the *proven boundary plus host injection*, by the headless harness, and — as of 0.8.1 — by a shipped DOM-free reference host (`headlessHost()`, exported from `@fundamental-engine/core`, #600), **not** by a second production renderer such as a native or WebGL backend. The architecture makes such a backend a host-implementation task rather than an engine rewrite; that it has not yet

been built is a fact, not a contradiction. The claim is "the engine is portable, and the boundary that makes it so is enforced," not "Fundamental ships on N renderers."

These limits do not weaken the central claim; they sharpen it. The contribution is an *enforced* separation and a *checkable* portability story, stated to exactly the precision the code supports.

1.8 7. Limitations

Beyond the two honest limits of §6.3 — the legacy element write-back path in `engine/field.ts` not yet routed through the `FeedbackRegistry` (Phase 5; #228), and the absence of a shipping non-DOM production renderer — the architecture carries two further caveats a preprint must name.

Ceremony has a cost. The six-phase scheduler and six registries impose real overhead on the author and the maintainer: a behavior must be decomposed into the right phase, state must be registered before it can be styled, feedback must go through one writer, and relationships must resolve to registered bodies. For a simple, flat interface this ceremony may not pay for itself — the same tradeoff the flagship names (§10) — and the lint rules that keep the contract honest are themselves a surface the author must learn. The architecture earns its keep on content with latent relational structure, not on a single button.

The portability argument is structural, not yet empirical across renderers. We argue portability from the boundary, the host seam, and the headless harness. We have not *run* the engine on a second production renderer, so the claim is an engineering argument about what the architecture admits, not a measured demonstration of N working backends. We are explicit that this is the form of the evidence.

Conformance proves forces, not every composition. The passport cross-check and the conformance harness prove individual forces and a set of composite experiments; they do not exhaustively prove every one of the 64 patterns' emergent behavior, only that each pattern references real vocabulary and that its constituent forces are individually conformant.

1.9 8. Discussion

Why host-driven separation matters for UI-behavior systems generally. The lesson generalizes well past Fundamental. Any system that computes *what an interface does* — animation engines, layout solvers, gesture recognizers, reactive runtimes, simulation-driven UIs — faces the same temptation to reach for `document`, `window`, and `requestAnimationFrame` inline, and the same three consequences when it does: the behavior cannot be tested without a browser, cannot be retargeted without a rewrite, and cannot be audited in isolation. A single injected host boundary dissolves all three at once. *Testability:* the behavior runs against a stub host, so it is unit-testable and, in Fundamental's case, the production physics runs headlessly and deterministically. *Retargetability:* a new render surface is a new host implementation, not a fork of the engine. *Auditability:* with the environment behind one interface and the participation contract expressed as registries and lint rules, a reviewer can check that the core touches no global, that no overlay mutates physics, that feedback writes only CSS variables, and that a behavior references only real vocabulary — by running tests, not by trusting prose.

The boundary as a forcing function for honesty. The empty-allowlist test does more than keep the core clean; it forces the *design* to name every environment dependency explicitly. You cannot casually add a `matchMedia` call deep in the engine — the test fails, and you are obliged to add the capability to `FieldHost` and implement it in the host. That friction is the point: it converts every new coupling to the environment into a deliberate, reviewable widening of a single, small interface. The same pattern recurs one layer down in the scheduler's read-phase guard, which is injected into the measurement registry as a single function so the registry never imports the scheduler.

Auditable behavior as a portability property. A subtle benefit of the conformance machinery is that it makes *behavior* portable, not just the engine. Because a pattern is validated against passported vocabulary

and its forces are proven by a headless harness, a pattern carries its own guarantees: drop it onto any host that implements `FieldHost` and the same honest metrics are computed and the same proven forces run. The supplied-vs-derived discipline (confidence supplied-only; resolution real) means those metrics never fabricate evidence — a property that holds *by construction across host targets*, because it lives in the renderer-agnostic core. This is the right posture for a system that asks designers to reason about *cause* rather than appearance: the cause is computed in a place that is provably independent of how it is drawn.

1.10 9. Conclusion

Fundamental stays portable by separating *what the field does* from *how the browser participates in it*, and connecting the two through exactly one injected host boundary. The renderer-agnostic core computes field, force, metric, and conformance logic against plain data and imports no DOM globals — a property not asserted but enforced by a boundary test that runs with an empty allowlist, so the core computes with no document present. On top of that boundary, a six-phase scheduler and six single-concern registries turn DOM participation into a disciplined, lintable contract that lets the field ride the browser's layout without thrash, and a pattern runtime backed by validation, passport cross-checks, a headless physics harness, and a supplied-vs-derived metric discipline turns interface behavior into a portable, auditable program. We have made the engineering case for portability — one core already driving a canvas, SVG overlays, CSS-variable feedback, and a headless harness, with a clear host- shaped path to WebGL, native, or a different document — and we have been candid about its bounds: a legacy element write-back path in `engine/field.ts` is still being migrated behind the platform registries (Phase 5; #228), and there is no shipping non-DOM production renderer yet. Portability is demonstrated by a proven boundary plus host injection, not by a second production renderer. The companion papers validate the paradigm this runtime serves in their own domains — reading (Paper 2), evidence and trust (Paper 3), reduced-motion equivalence (Paper 4), pattern authoring (Paper 6), data binding (Paper 7), and explainable diagnostics (Paper 8) — and the flagship (Paper 1) names the paradigm itself.

1.11 Appendix A. Reproducibility

Every architectural claim in this paper is checkable against the repository. The load-bearing anchors:

- **The host boundary.** The `FieldHost` interface: `packages/core/src/engine/host.ts`. The browser adapter: `packages/dom/src/browser-host.ts` (`browserHost()`). The shipped DOM-free reference host: `headlessHost()`, exported from `@fundamental-engine/core` (#600). The mandatory-host entry point: `packages/core/src/engine/field.ts` (`createField` throws without `opts.host`).
- **The proof.** The empty-allowlist boundary test: `packages/core/src/engine/dom-boundary.test.ts` (`const ALLOW = new Set<string>()`). The legacy element write-back path it documents: `packages/core/src/engine/field.ts` (the `--field-density` / `transform` / `data-*` writes and the `feedbackSink` seam).
- **The scheduler and registries.** `packages/dom/src/schedule.ts` (`FrameScheduler`, `PHASES`, `READ_PHASES`, `readGuard`, `assertPhase`), `platform.ts` (`createFieldPlatform`), and the six registries `measurement.ts`, `state.ts`, `feedback.ts`, `relationships.ts` (with `scanRelationships` and the resolved/unresolved split), `visual-bindings.ts`, `overlays.ts`.
- **The self-audit.** `packages/dom/src/lint.ts` (`lintPlatform` and its seven pure rules).
- **The pattern runtime and conformance.** `packages/dom/src/apply-recipe.ts` (`applyRecipe`), `packages/core/src/recipes/schema.ts` (`validateRecipe`, the `OTHER_LANE` cross-lane guard), `packages/core/src/recipes/catalog.ts` (the 64 patterns as a conformance fixture), `packages/core/src/contracts` (`PASSPORTS`, `validatePassports`), `packages/core/src/contracts/passport.test.ts` (the cross-check test), and `packages/core/src/conformance/run.ts` (the headless physics harness).
- **The metric discipline.** `packages/dom/src/metrics.ts` (computed vs supplied-only; `confidence` optional; the removed `confidence = resolvedRatio` default).

- **The merged fixes.** Confidence supplied-only: #220. Real relationship resolution: #222. Memory reclassified as a semantic metric: #223. The renderer-agnostic engine via injected `FieldHost`: #192.

The canonical design documents corroborate the framing: `docs/canonical/Fundamental-platform-architecture.md` (the package hierarchy, the `FrameScheduler`, the six registries, `lintPlatform`, Phase D), and `docs/canonical/system-contracts.md` (§20 Platform Contract, §21 Registry Contract, §22 Scheduler Contract, §23 Platform-Lint Contract, §24 the Phase-D runtime-unification note that states the legacy-write-back caveat).

1.12 Appendix B. Conversion notes (markdown → preprint)

Notation is kept LaTeX-compatible (inline math and fenced code blocks translate directly). Figures referenced in prose but not yet drawn — the package dependency graph and host seam (§3.1–3.2), the six-phase scheduler timeline and read/write discipline (§4.1–4.2), the registry-to-phase map (§4.3), and the recipe-to-platform compilation pipeline (§5.1) — are produced at conversion time from the prose descriptions. External citations and bibliography keys in `references.md` must be resolved and verified before submission. Cross-references to sibling papers use their series numbers; replace with citation keys at conversion time.

1.13 Citation coverage

The following external references are cited in the text and have been located in `references.md` — never fabricated:

- **Retained-mode vs immediate-mode UI** (§2) — the architectural distinction between a persistent object model redrawn from state and per-frame re-issued draw calls.
- **Renderer-agnostic / headless UI frameworks** (§2) — engines that compute against an abstract scene and bind a backend late, and behavior/state libraries that prescribe no markup.
- **Hexagonal architecture / ports and adapters** (§2) — the domain-core-behind-ports pattern of which `FieldHost` is an instance.
- **Reactive-runtime scheduling / read-write batching** (§2) — the discipline of batching DOM reads and writes to avoid layout thrash, of which the six-phase scheduler is a named form.
- **Testability of UI logic / separation of concerns** (§2) — the argument that behavior is testable in proportion to its separation from the renderer.